

Семсічко Лідія

## Task 1 - Базова архітектура мікросервісів

Git hub:

[https://github.com/LidaSemsichko/apz\\_homeworks.git](https://github.com/LidaSemsichko/apz_homeworks.git)

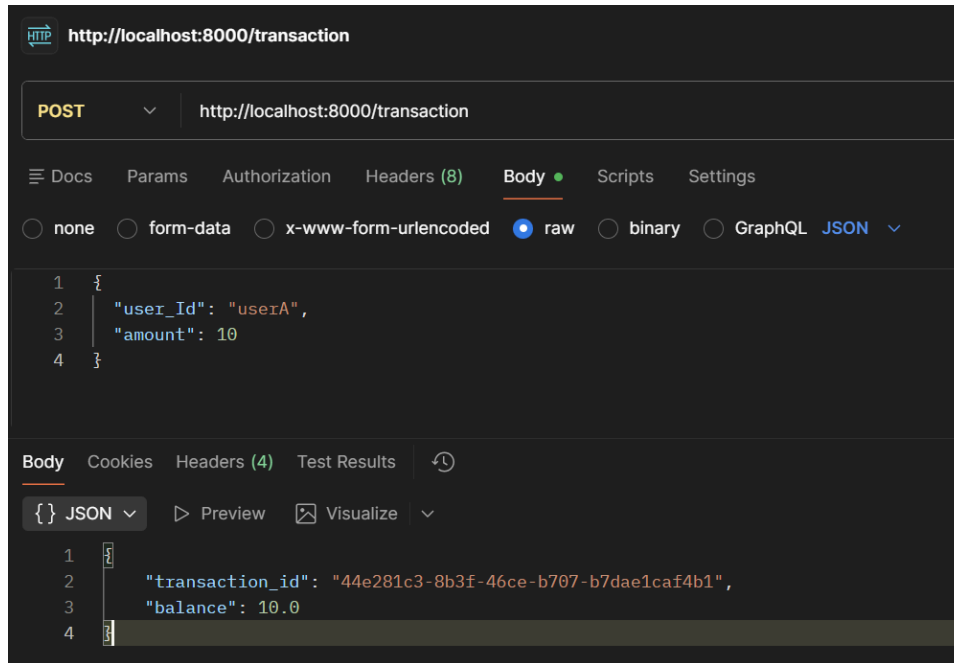
Гілка: micro\_basics

Запуск через Docker (якно зручніше, ніж відкривати 4 різних термінали)

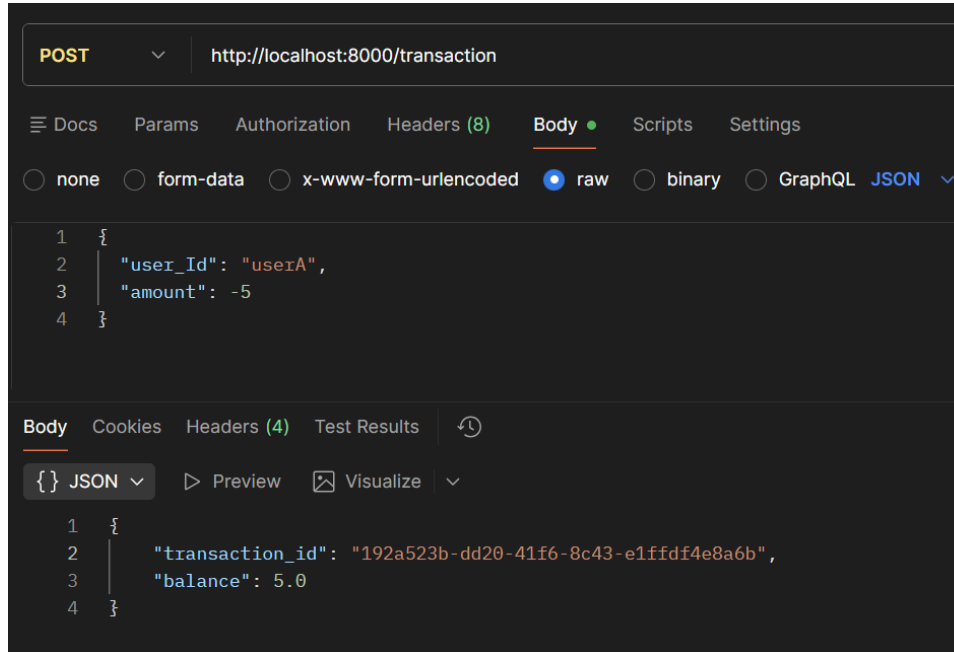
```
[+] Running 3/3
✓ Container apz_homeworks-logging-service-1 Recreated
✓ Container apz_homeworks-counter-service-1 Recreated
✓ Container apz_homeworks-facade-service-1 Recreated
Attaching to counter-service-1, facade-service-1, logging-service-1
logging-service-1 | INFO: Started server process [1]
logging-service-1 | INFO: Waiting for application startup.
counter-service-1 | INFO: Started server process [1]
logging-service-1 | INFO: Application startup complete.
counter-service-1 | INFO: Waiting for application startup.
logging-service-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
counter-service-1 | INFO: Application startup complete.
counter-service-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
facade-service-1 | INFO: Started server process [1]
facade-service-1 | INFO: Waiting for application startup.
facade-service-1 | INFO: Application startup complete.
facade-service-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

## Мануальне тестування

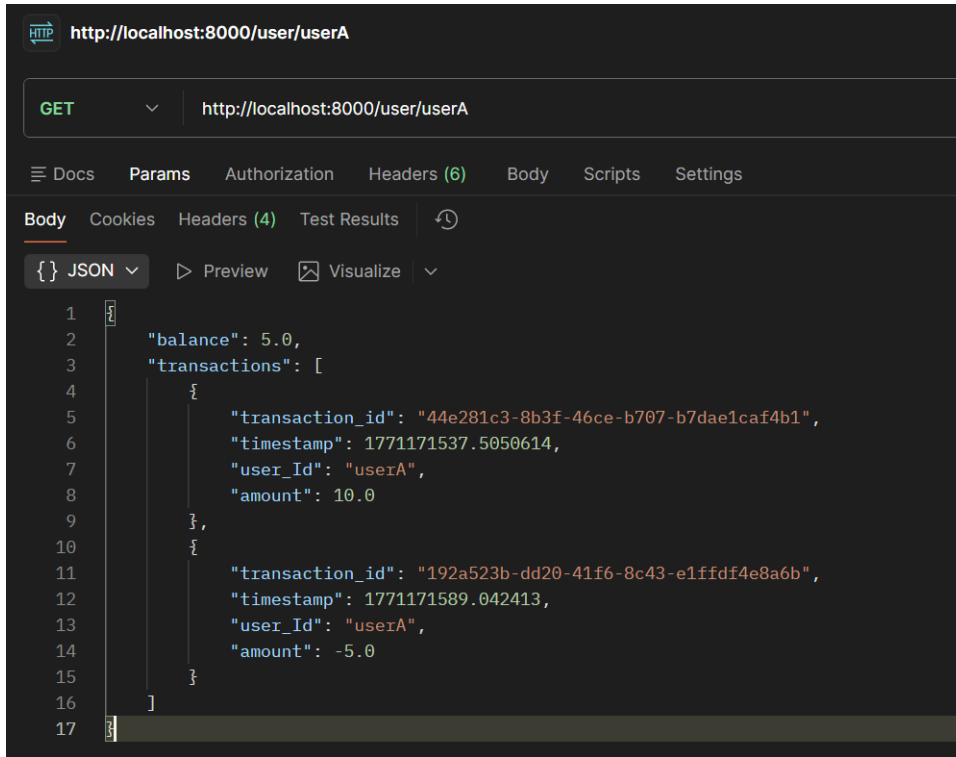
- Просто транзакція з додатною сумою. Видно, що додається новий профіль, сума правильна.



- Від'ємна. Те ж саме, арифметика працює

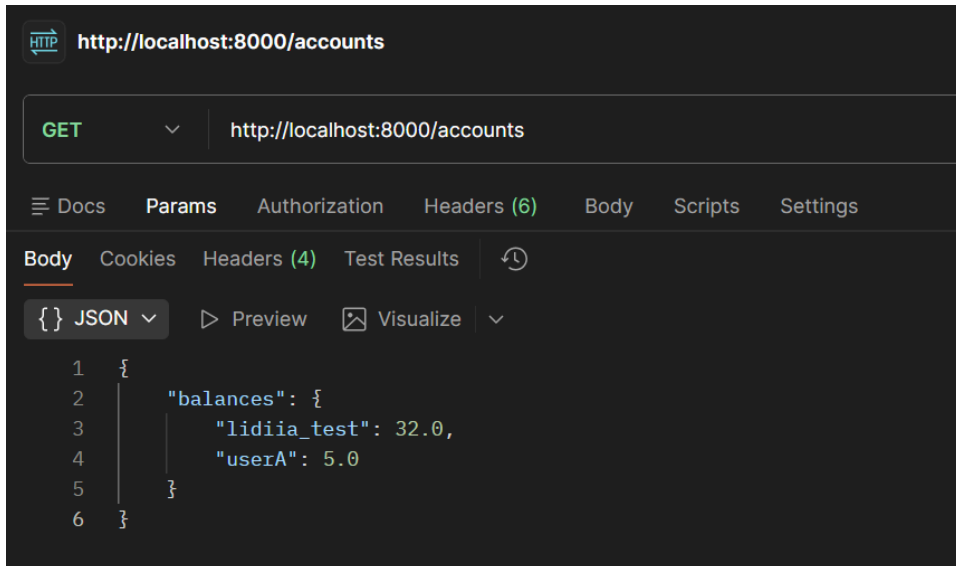


- Перевірка аккаунту. Обидві транзакції видно в аккаунті, все добре



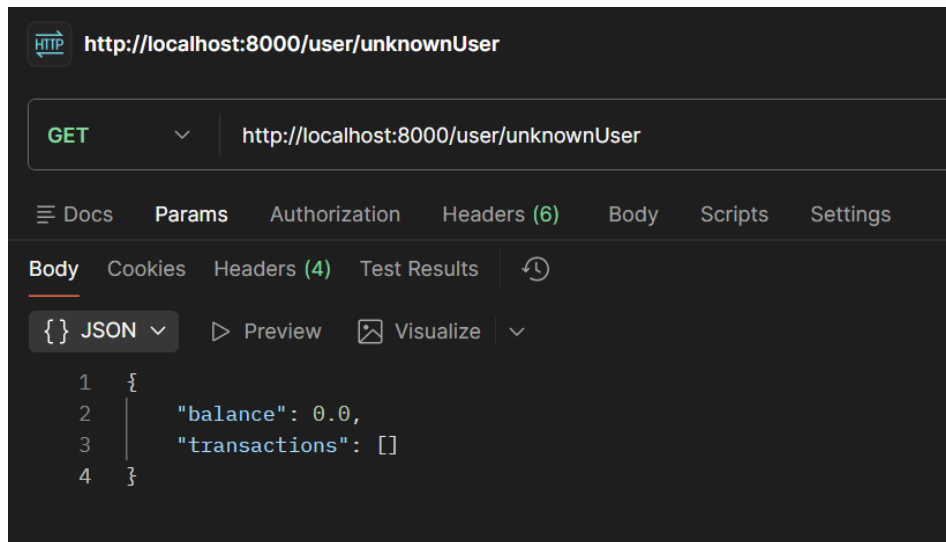
```
1 {
2   "balance": 5.0,
3   "transactions": [
4     {
5       "transaction_id": "44e281c3-8b3f-46ce-b707-b7dae1caf4b1",
6       "timestamp": 1771171537.5050614,
7       "user_id": "userA",
8       "amount": 10.0
9     },
10    {
11      "transaction_id": "192a523b-dd20-41f6-8c43-e1ffdf4e8a6b",
12      "timestamp": 1771171589.042413,
13      "user_id": "userA",
14      "amount": -5.0
15    }
16  ]
17 }
```

- Додала ще один аккаунт та перевірила список аккаунтів.

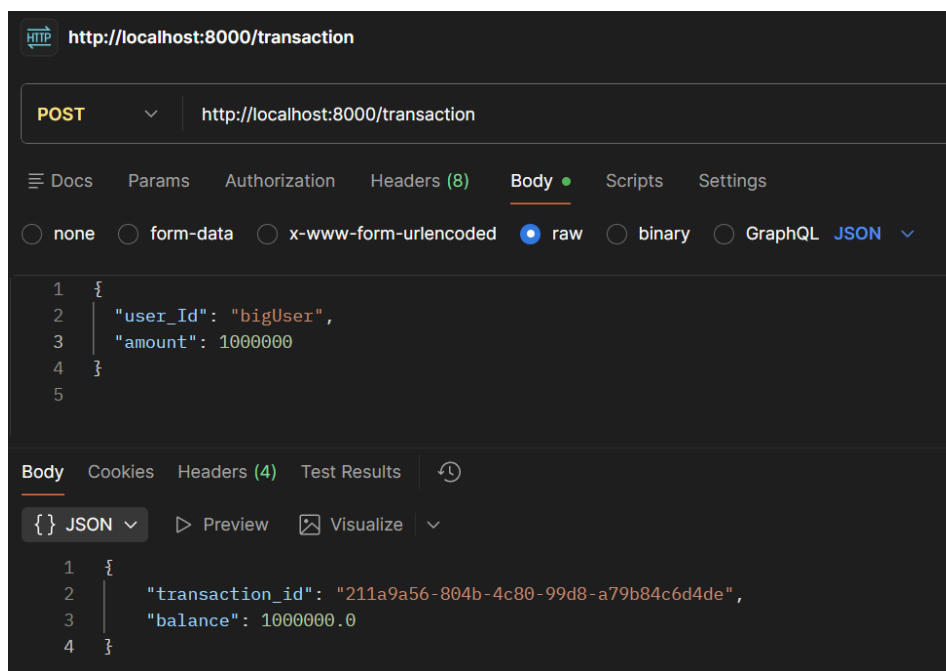


```
1 {
2   "balances": {
3     "lidiia_test": 32.0,
4     "userA": 5.0
5   }
6 }
```

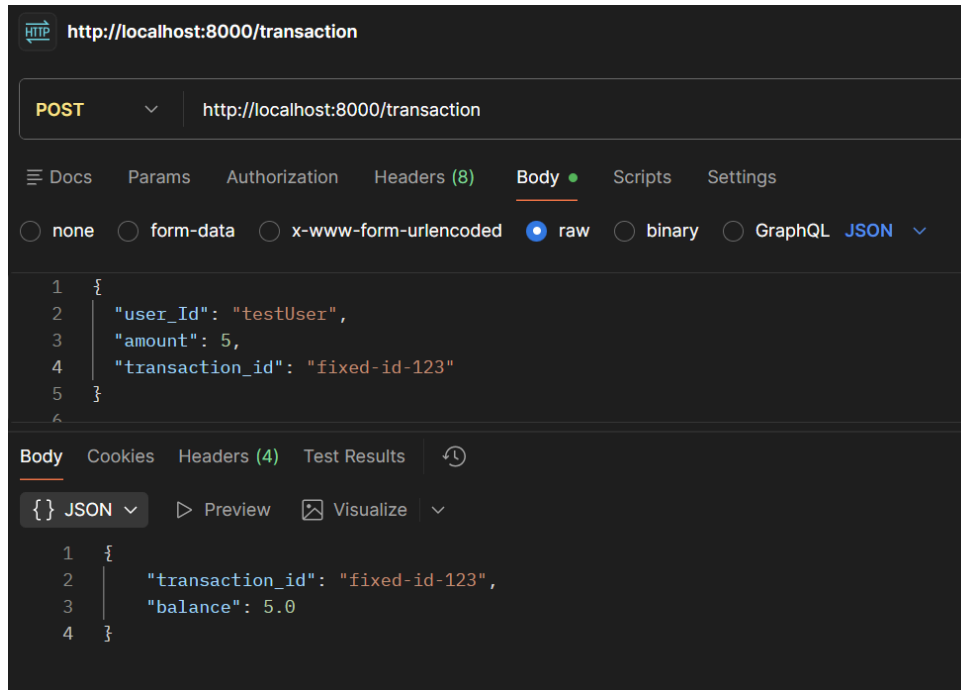
- Невідомий юзер. Тут теж все коректно, Баланс на нулі, транзакцій немає



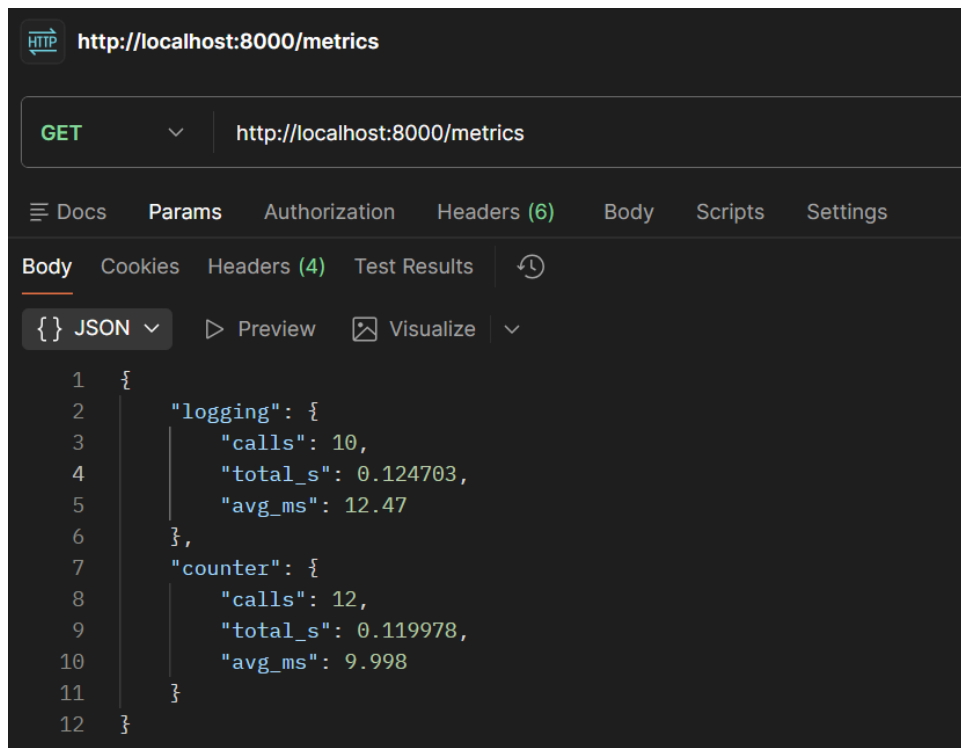
- Велика сума. Додалось в аккаунт правильно, без затримок



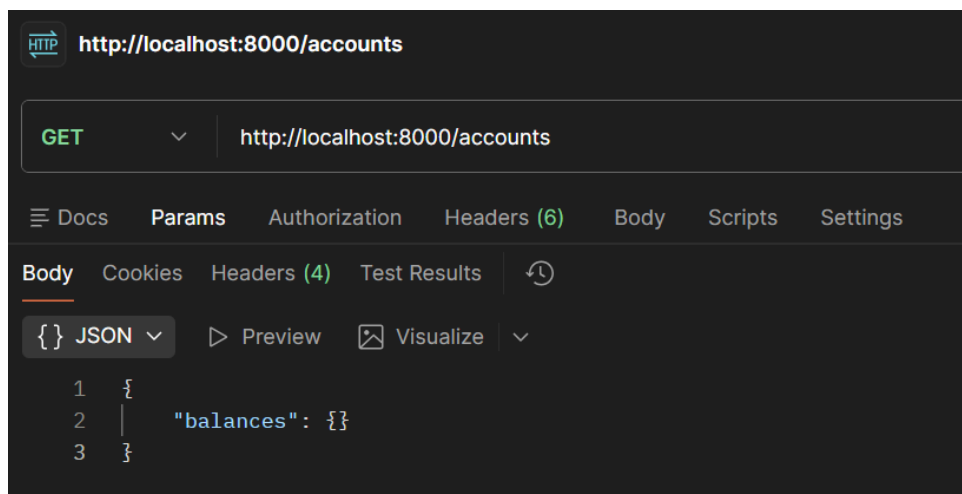
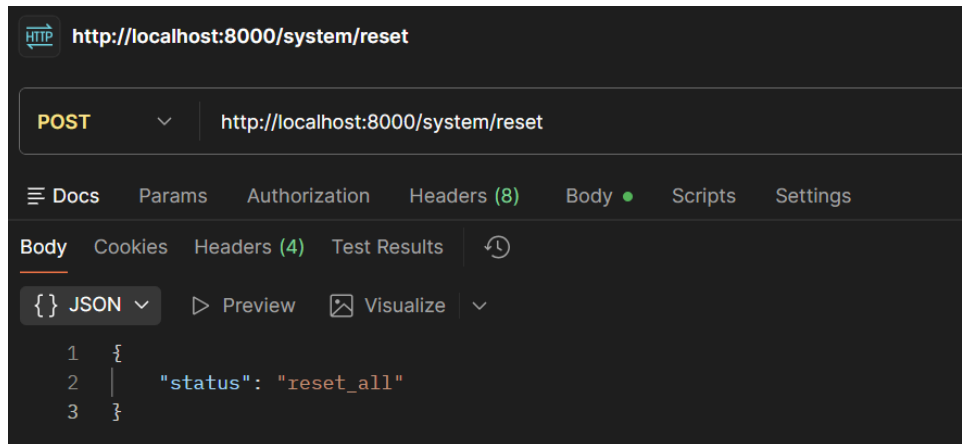
- Перевірка ідемпотентності (якщо дві транзакції з однаковим ід, то пройде лише одна з них)



- Перевірка метрик



- Reset. Чітко бачимо, що всі аккаунти та транзакції видалились.

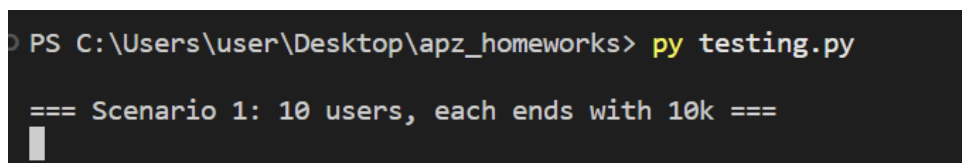


В ході мануального тестування проблем не було виявлено. Тому переходимо до лод тестів.

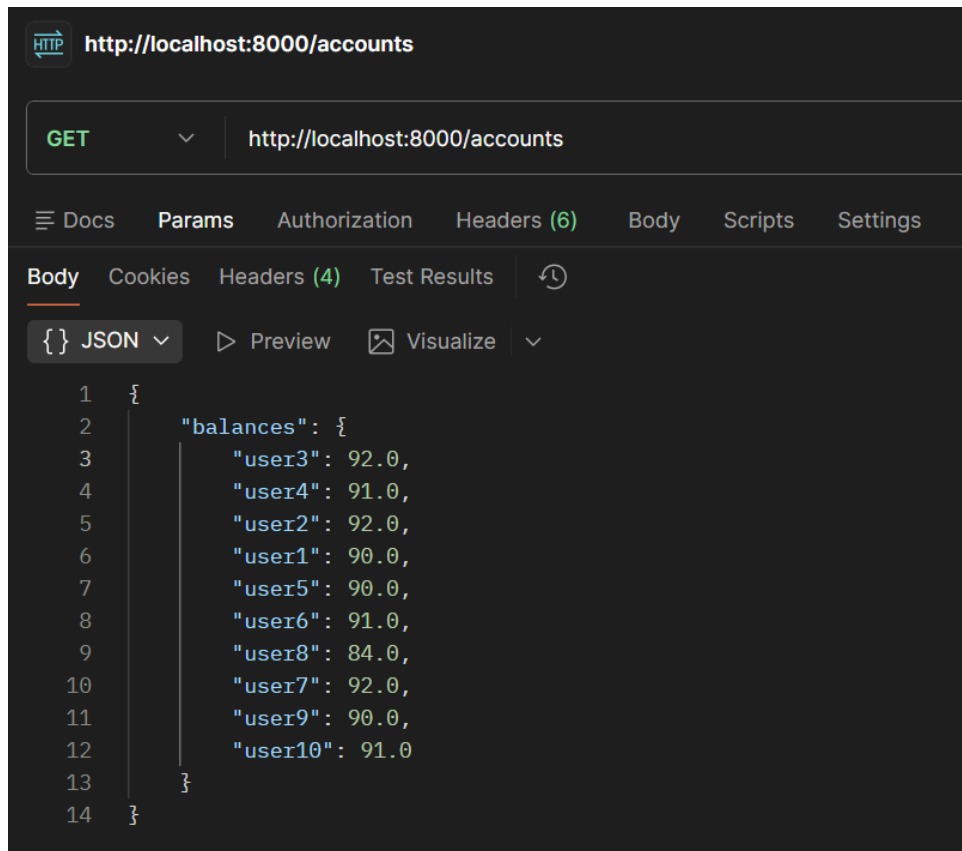
## Лод тестування

### Сценарій 1

Запускаємо пайтон скрипт

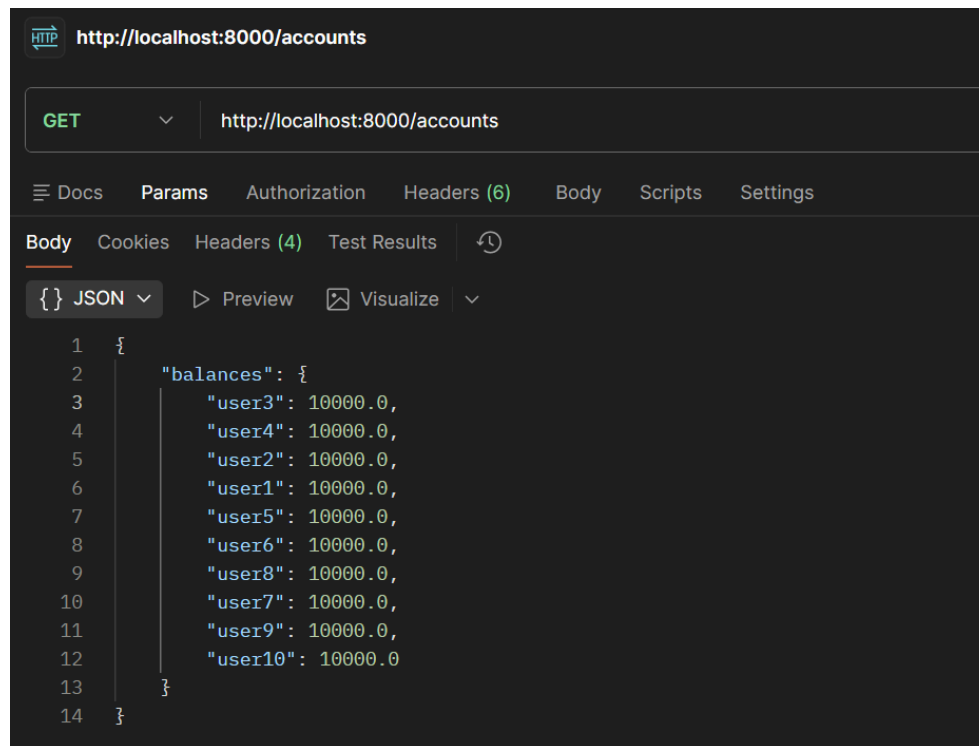


У Postman можна викликати /accounts та подивитись чи все у нас створилось добре та як начисляється баланс



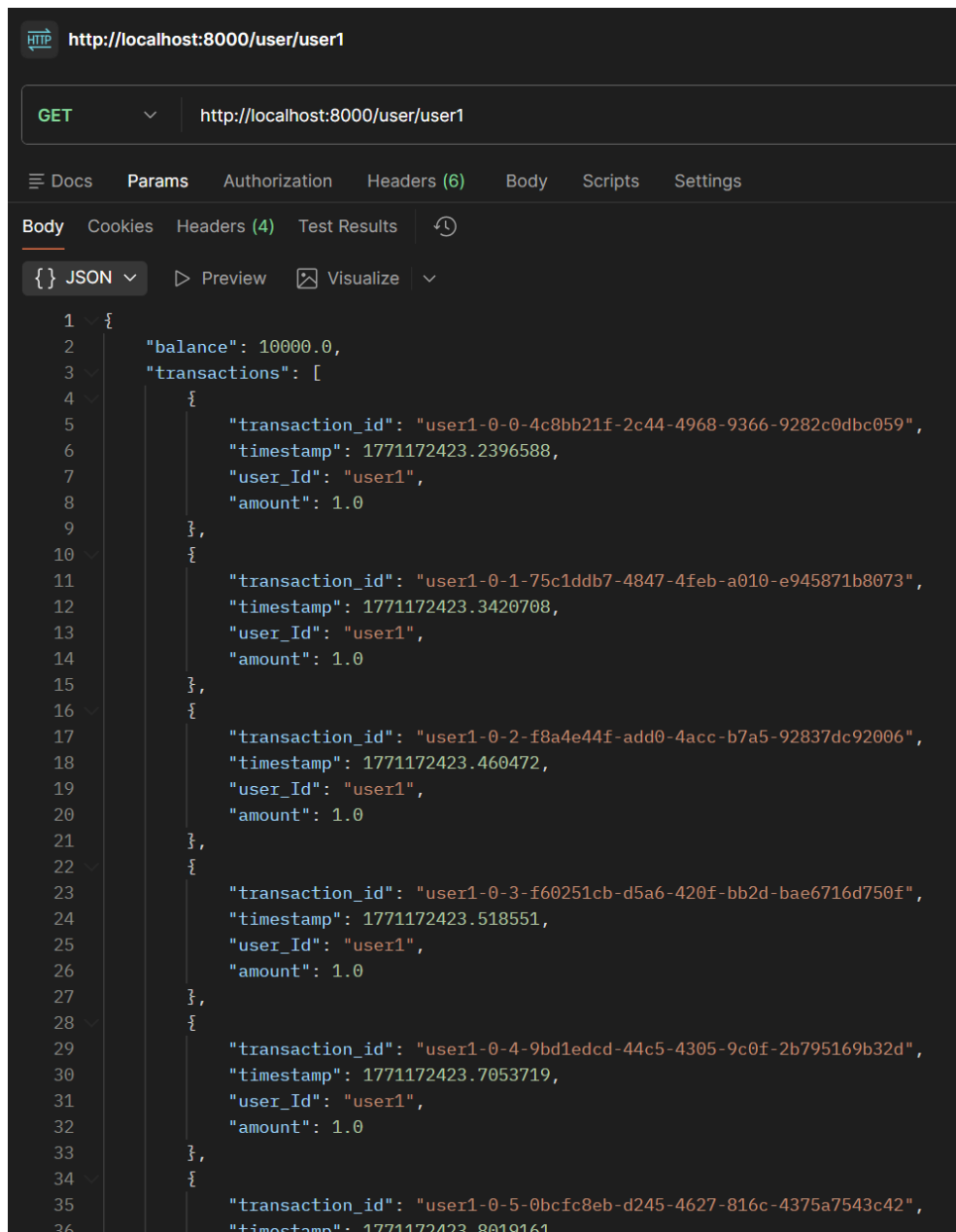
Тест закінчився:

Бачимо у Postman, що на всіх балансах правильна сума





Транзакції кожного юзера можна побачити окремо.



Логи операції отримуємо з виводу пайтон скрипта

```
=== Scenario 1: 10 users, each ends with 10k ===
Total requests: 100000
Total time (s): 1315.100
RPS: 76.04
Facade remote call timing (accumulated): {'logging': {'calls': 100040, 'total_s': 1501.260127, 'avg_ms': 15.007}, 'counter': {'calls': 100056, 'total_s': 1451.164342, 'avg_ms': 14.504}}
```

У першому сценарії система обробила 100 000 транзакцій за 1315 секунд, що відповідає середній пропускну здатності приблизно 76 запитів за секунду (можна на екрані вище побачити)

Кожна клієнтська транзакція ініціює два внутрішні HTTP-виклики, до logging-service та counter-service. Таким чином, загальна кількість внутрішніх HTTP-операцій перевищила 200 000. З урахуванням паралельного виконання запитів фактичне навантаження на систему становило понад 150 HTTP-викликів за секунду.

Середній час одного виклику до logging-service становив приблизно 15 мс, до counter-service - близько 14.5 мс. Обидва сервіси продемонстрували близькі показники затримки, що свідчить про рівномірний розподіл навантаження між ними.

Під час реалізації самого цього тестування було перевірено декілька варіантів організації обробки запитів:

1. Без механізму блокування у counter-service. У цьому випадку при паралельному оновленні одного і того самого рахунку виникала race condition. Частина інкрементів втрачалась, і фінальний баланс становив 6 000–8 000 замість очікуваних 10 000. Це пов'язано з тим, що одночасні операції читання-запису значення балансу перезаписували одна одну. Проаналізувала це та виправила.
2. Також розглядала варіант без повторної відправки запитів у клієнті. Під високим навантаженням частина HTTP-запитів завершувалась помилками або тайм-аутами. У таких випадках транзакції не доходили до сервісів або не підтверджувались, що призводило до зменшення фінального балансу. У результаті замість 10 000 транзакцій на рахунку знову фіксувалось то по 5тис, то по 9тис.
3. Без ідемпотентності транзакцій. При додаванні механізму повторної відправки запитів без контролю унікальності transaction\_id виникала протилежна проблема, можливе

подвійне застосування однієї й тієї самої транзакції, що призводило до завищення балансу. Це теж вирішила, тест на ідемпотентність є зверху.

## Висновок

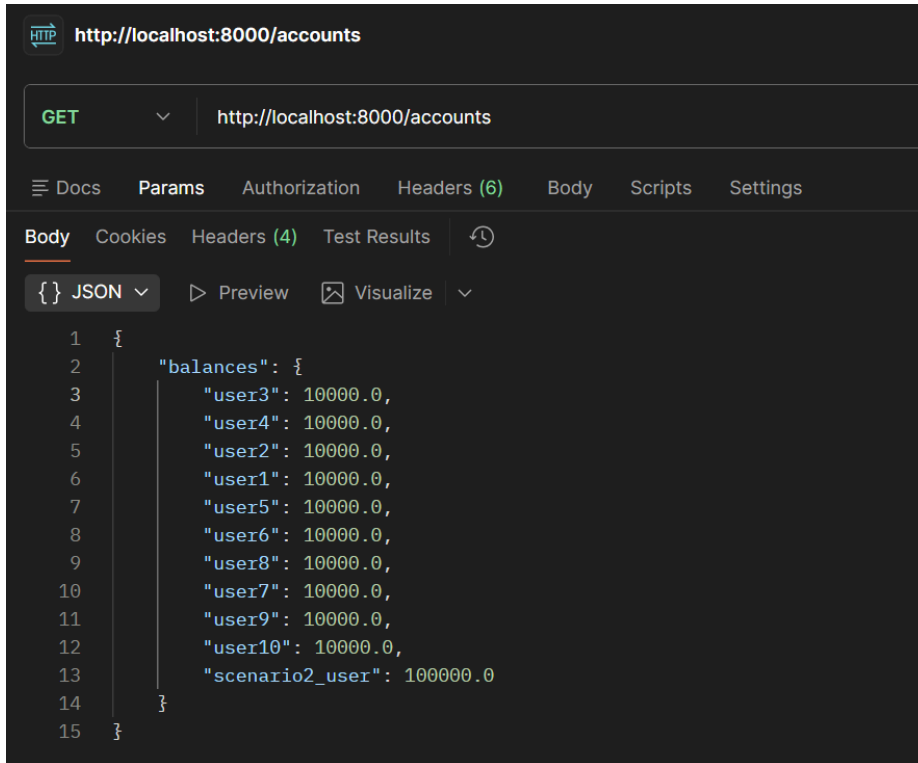
Отримані результати показали, що система нормально працює навіть при великій кількості паралельних запитів. Баланси рахунків у всіх сценаріях повністю відповідали очікуваним значенням, тобто транзакції не губились і не дублювались.

Хоча швидкість обробки запитів обмежується тим, що між сервісами використовується HTTP-взаємодія через Docker, а також ресурсами локального комп'ютера, система поводить себе стабільно і передбачувано під навантаженням. Порахувало все нормально, всі запити дійшли, ми нікого не забули, успіх.

## Сценарій 2

Запускаємо тим самим пайтон скриптом, тільки треба розкоментувати другий тест)

Після завершення (не робила reset після першого), бачимо, що у нас додався новий юзер, та його баланс становить 100 000, як і мало бути.



Логи знову бачимо в терміналі

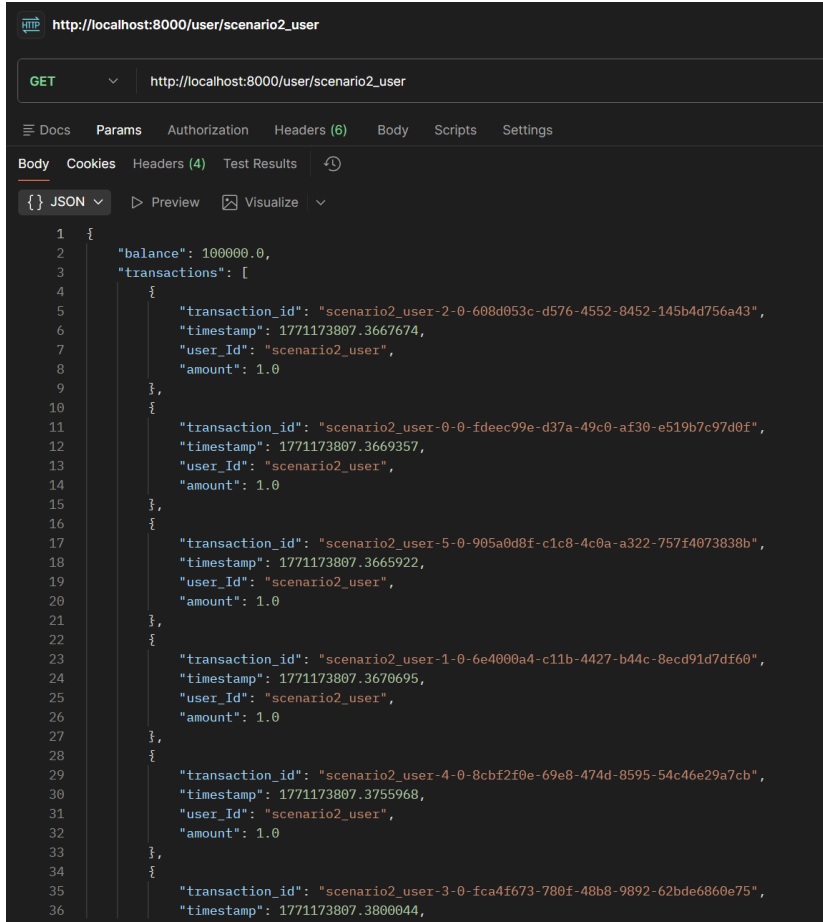
```
=== Scenario 2: 1 user, ends with 100k ===
Total requests: 100000
Total time (s): 1834.415
RPS: 54.51
Facade remote call timing (accumulated): {'logging': {'calls': 100051, 'total_s': 1927.659614, 'avg_ms': 19.267}, 'counter': {'calls': 100210, 'total_s': 1841.403406, 'avg_ms': 18.375}}
```

У другому сценарії всі 100 000 транзакцій виконувались для одного і того самого рахунку. Це призвело до значного зростання конкурентного доступу до спільного ресурсу. Загальний час виконання склав 1834 секунди, що відповідає пропускну здатності 54 запити/сек, що приблизно на 30% менше, ніж у першому сценарії.

Середній час виклику до logging-service зріс до ~19 мс, а до counter-service до ~18 мс. Основною причиною зниження продуктивності є використання механізму блокування у counter-service, який гарантує відсутність race condition, але серіалізує доступ до одного й того самого балансу.

Незважаючи на зниження пропускну здатності, система забезпечила повну коректність фінального результату, баланс

рахунку склав рівно 100 000, що підтверджує відсутність втрати транзакцій. Власне якщо аналізувати реальні системи, то підвищення часу очікування на декілька секунд не є таким важливим, як правильність підрахунків, тому на мою думку це правильне рішення.



```
1 {
2   "balance": 100000.0,
3   "transactions": [
4     {
5       "transaction_id": "scenario2_user-2-0-608d053c-d576-4552-8452-145b4d756a43",
6       "timestamp": 1771173807.3667674,
7       "user_id": "scenario2_user",
8       "amount": 1.0
9     },
10    {
11      "transaction_id": "scenario2_user-0-0-fdeec99e-d37a-49c0-af30-e519b7c97d0f",
12      "timestamp": 1771173807.3669357,
13      "user_id": "scenario2_user",
14      "amount": 1.0
15    },
16    {
17      "transaction_id": "scenario2_user-5-0-905a0d8f-c1c8-4c0a-a322-757f4073838b",
18      "timestamp": 1771173807.3665922,
19      "user_id": "scenario2_user",
20      "amount": 1.0
21    },
22    {
23      "transaction_id": "scenario2_user-1-0-6e4000a4-c11b-4427-b44c-8ecd91d7df60",
24      "timestamp": 1771173807.3670695,
25      "user_id": "scenario2_user",
26      "amount": 1.0
27    },
28    {
29      "transaction_id": "scenario2_user-4-0-8cbf2f0e-69e8-474d-8595-54c46e29a7cb",
30      "timestamp": 1771173807.3755968,
31      "user_id": "scenario2_user",
32      "amount": 1.0
33    },
34    {
35      "transaction_id": "scenario2_user-3-0-fca4f673-780f-48b8-9892-62bde6860e75",
36      "timestamp": 1771173807.3800044,
```

Ну і всі транзакції теж видно))

Загалом весь вивід можна побачити одразу в терміналі

```
facade-service-1 | INFO: 172.19.0.1:37606 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36450 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44876 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37568 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36462 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44892 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37646 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36476 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44944 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37620 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36450 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44876 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37586 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36462 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44892 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37652 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36476 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44944 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37602 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36450 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44876 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37584 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36462 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44944 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37632 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36450 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44876 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37604 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36462 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44944 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37568 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36450 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44876 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37606 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36462 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44892 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37586 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36476 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44944 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37620 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36450 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44876 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37646 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36462 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44892 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37602 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36476 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44944 - "POST /log HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.19.0.1:37652 - "POST /transaction HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.19.0.4:36450 - "POST /apply HTTP/1.1" 200 OK
logging-service-1 | INFO: 172.19.0.4:44876 - "POST /log HTTP/1.1" 200 OK
Gracefully stopping... (press Ctrl+C again to force)
[+] Stopping 3/3
✓ Container apz_homeworks-facade-service-1 Stopped
✓ Container apz_homeworks-counter-service-1 Stopped
✓ Container apz_homeworks-logging-service-1 Stopped
```